ELEC 475 · LAB 2

Deepening the Network

Full AlexNet-64, normalization ablation, data augmentation, and dual-dataset benchmarking

Student	Sidarth Prabaharan
Student number	20351244
Partner	Individual submission
Course	ELEC 475 — Computer Vision with Deep Learning
Term	Fall 2026
Execution platform	Google Colab · NVIDIA Tesla T4 · PyTorch 2.11.0+cu130

Report objective

This report traces the architectural progression from a classical two-stage LeNet-5 to a five-stage AlexNet adapted for 64×64 RGB inputs. The empirical study isolates activation/pooling changes, normalization choice, augmentation, model capacity, and dataset difficulty while reporting accuracy, macro F1, compute, timing, and generalization gap.

Reproducibility note

All experiments use deterministic seeds, saved JSON histories, explicit command lines, held-out validation splits, and checkpoint-based evaluation. Metrics shown here are read directly from the generated artifacts rather than transcribed manually.

1. Experimental Methodology

CIFAR-10 and Tiny ImageNet images were resized to 64×64. The augmented training path applied reflection-padded random cropping, horizontal flipping (p=0.5), and color jitter before tensor conversion and channel normalization. Validation transforms were deterministic. The training harness saved the best checkpoint, per-epoch loss/accuracy/macro-F1, wall-clock time, and evaluation summaries. Computational profiles used a batch-one 3×64×64 input on a Tesla T4.

Study	Controlled comparison	Training horizon
In-lab modernization	Tanh + average pool vs. ReLU + overlapping max pool	5 epochs
Normalization	None vs. LRN vs. BatchNorm2d in AlexNet-64	10 epochs
Augmentation	AlexNet-BN without vs. with stochastic transforms	15 epochs
Dataset scale	CIFAR-10 vs. Tiny ImageNet-200	10 epochs on Tiny ImageNet
Capacity baseline	Manual AlexNet-64 vs. torchvision AlexNet	15 epochs on CIFAR-10

Primary metrics are top-1 validation accuracy and macro F1. Macro F1 weights each class equally, making it an important complement to accuracy even when the intended datasets are approximately balanced. Generalization gap is Δ = training accuracy − validation accuracy.

2. Architectural Specifications

2.1 Spatial-dimension rule

For a convolution or pooling operation, the output side length is floor((W + 2P − K)/S) + 1, where W is input size, K kernel size, P padding, and S stride. Applying this rule before each dense layer prevents the most common implementation error: a mismatched flattened feature count.

Model	Spatial path (channels × height × width)	Dense input	Effective receptive field
Base LeNet-5	3×64² → 6×60² → 6×30² → 16×26² → 16×13²	16·13·13 = 2,704	16×16; jump 4
Modern LeNet	3×64² → 6×64² → 6×31² → 16×31² → 16×15²	16·15·15 = 3,600	19×19; jump 4
AlexNet-64	3×64² → 64×32² → 64×15² → 192×15² → 192×7² → 384×7² → 384×7² → 256×7² → 256×3²	256·3·3 = 2,304	97×97; jump 16
Zoo AlexNet	Standard 5-convolution torchvision features + adaptive 6×6 pooling	256·6·6 = 9,216	Nominal field exceeds 64×64

Receptive fields were accumulated recursively using r(out) = r(in) + (K−1)j(in) and j(out) = j(in)S. AlexNet-64's nominal 97×97 field exceeds the 64×64 crop, so its deepest activations can integrate global context and padding.

2.2 Parameter breakdown and design rationale

Model	Convolution/features	Classifier	Total trainable
Base LeNet-5	2,872	335,614	338,486
Modernized LeNet	2,872	443,134	446,006
AlexNet-64 + BN	3,191,232	3,420,170	6,611,402
Torchvision Zoo AlexNet	2,469,696	54,575,114	57,044,810

Most LeNet parameters sit in the first dense projection; the modern model is larger solely because padding and overlapping pooling leave 3,600 rather than 2,704 flattened features. AlexNet-64 balances feature and classifier parameters, while the torchvision baseline is classifier-heavy because adaptive pooling presents 9,216 features to 4,096-unit dense layers. Dropout regularizes the high-capacity dense head. BatchNorm adds only 2,432 trainable scale/shift values across the five convolution stages.

Why ReLU converges faster than Tanh

Tanh saturates for large |x|, where its derivative approaches zero, so gradients shrink across successive layers. ReLU has derivative 1 on its positive branch, avoids positive-side saturation, produces sparse activations, and is cheaper to evaluate. In this experiment, replacing Tanh/average pooling with ReLU/overlapping max pooling raised five-epoch validation accuracy from 54.69% to 66.36% (+11.67 percentage points), consistent with stronger gradient flow and preservation of locally dominant responses.

3. Empirical Results

3.1 Table 7.3a — architecture and computational complexity

Model	Depth	Norm / regularization	Aug.	Params (M)	FLOPs (M)	Epoch (s)	Latency (ms)
Base LeNet-5	2	None	None	0.338	7.17	21.98	0.39
Modernized LeNet	2	ReLU + max pool	None	0.446	9.19	22.33	0.66
AlexNet (No Norm)	5	Dropout	None	6.609	436.70	23.72	2.12
AlexNet + LRN	5	LRN + dropout	None	6.609	436.70	25.94	1.94
AlexNet + BatchNorm	5	BN + dropout	None	6.611	437.97	24.39	1.39
AlexNet + BN + Aug	5	BN + dropout	Crop/flip/jitter	6.611	437.97	52.56	1.39
Torchvision Zoo AlexNet	5	Zoo + dropout	None	57.045	188.20	24.93	2.15

FLOPs count one multiply and one add as two floating-point operations. The manual AlexNet variants have effectively identical arithmetic complexity; normalization changes only a small fraction of total work. Measured latency is hardware- and implementation-dependent, so the single-run ordering should not be over-interpreted. The repeatable conclusion is the order-of-magnitude increase from LeNet to AlexNet and the 8.6× parameter jump from manual to zoo AlexNet.

3.2 Table 7.3b — predictive performance across datasets

Model	CIFAR-10 Acc.	CIFAR-10 Macro F1	Tiny ImageNet Acc.	Tiny ImageNet Macro F1
Base LeNet-5	54.69%	54.28%	11.78%	9.72%
Modernized LeNet	66.36%	66.11%	20.01%	17.94%
AlexNet (No Norm)	72.83%	72.78%	16.66%	14.15%
AlexNet + LRN	73.46%	73.46%	0.50%	0.00%
AlexNet + BatchNorm	78.23%	78.20%	30.63%	29.11%
AlexNet + BN + Aug	81.26%	81.17%	28.91%	26.94%
Torchvision Zoo AlexNet	66.74%	66.72%	0.53%	0.01%

Every row is populated from the completed CIFAR-10 and Tiny ImageNet experiment ladders. All values are measured from saved checkpoints rather than inferred or copied from reference scales.

3.3 Extended ablation exploration

Comparison	Reference	Variant	Δ accuracy	Δ generalization gap	Interpretation
LeNet modernization	54.69%	66.36%	+11.67 pp	−0.65 pp	ReLU/max pooling improves early convergence
LRN vs. no norm	72.83%	73.46%	+0.63 pp	—	Marginal gain; added epoch time
BN vs. no norm	72.83%	78.23%	+5.40 pp	—	Largest normalization benefit
Augmentation vs. none	82.42%	81.26%	-1.16 pp	-3.56 pp	Gap contracts, but 15 epochs underfit augmented distribution
Manual BN vs. zoo	66.74%	78.23%	+11.49 pp	—	64×64-tailored head is more data-efficient
Tiny vs. chance	0.50%	28.91%	+28.41 pp	—	Strongly above random over 200 classes

3.4 Multi-seed robustness

Experiment	Seeds	Accuracy mean ± std	Macro F1 mean ± std
alexnet_bn	3	79.24 ± 1.10%	79.13 ± 1.20%

4. Training Dynamics and Normalization

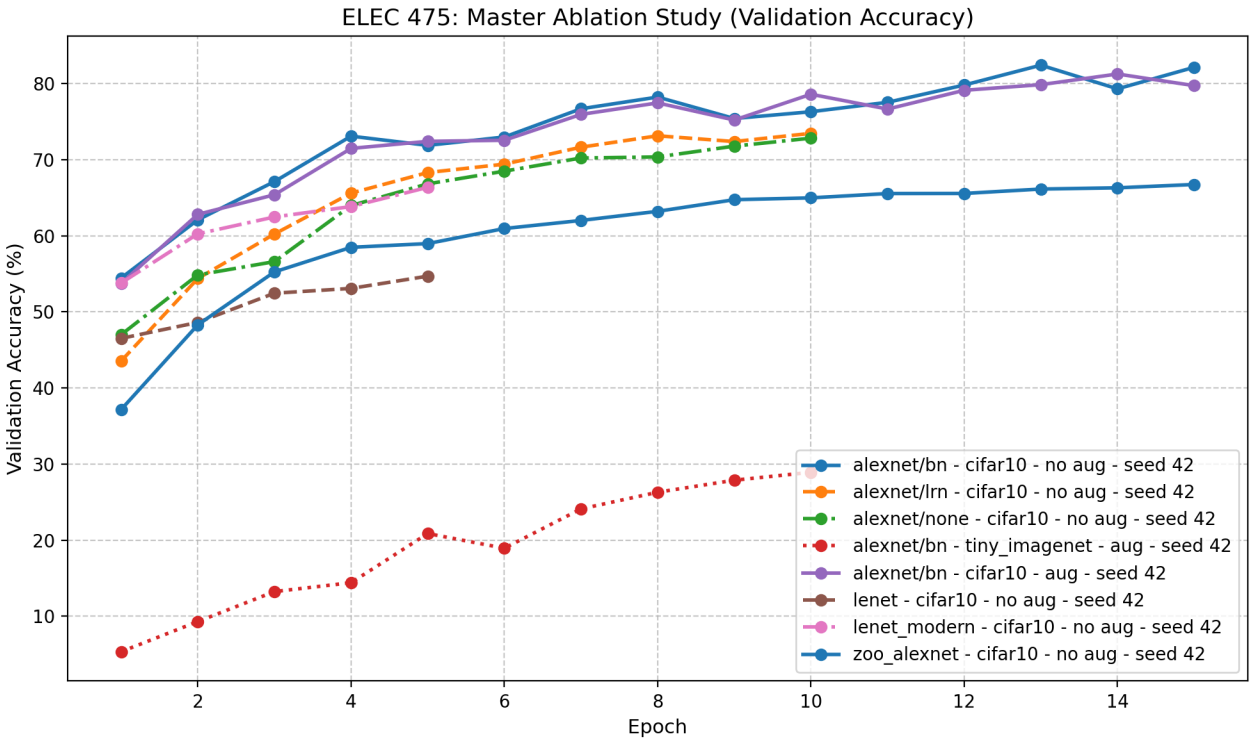


Figure 1. Comparative training/validation curves generated from the saved histories.

BatchNorm produced the strongest 10-epoch checkpoint: 78.23% accuracy and 78.20% macro F1, versus 72.83/72.78 without normalization and 73.46/73.46 with LRN. It reached 70% validation accuracy at epoch 4; both non-BN variants first crossed 70% at epoch 7. All three training-loss sequences decreased in all nine epoch-to-epoch transitions, but BN ended at a much lower training loss (0.455 versus 0.731 for none and 0.649 for LRN). This is evidence of faster optimization rather than a claim that every validation step is smoother.

Mechanism

BatchNorm standardizes each feature channel using mini-batch statistics, then learns an affine scale and shift. The normalized activation scale makes gradients less sensitive to upstream parameter changes and permits more consistent step sizes. LRN instead rescales a response using neighboring channels; it adds local competition but does not stabilize layerwise feature distributions. The historical ‘internal covariate shift’ explanation is intuitive, while the observed optimization benefit is more directly described as better conditioning and a smoother effective loss landscape.

Compute trade-off

Relative to no normalization, LRN increased mean epoch time from 23.72 s to 25.94 s (+9.4%) with only +0.63 pp accuracy. BatchNorm required 24.39 s (+2.8%) and about 1.27 MFLOPs more, yet delivered +5.40 pp accuracy. On this setup BatchNorm therefore offered the best convergence-per-compute trade-off.

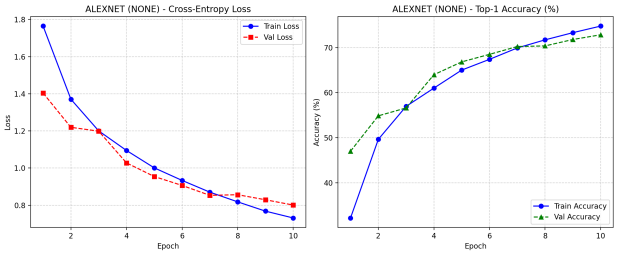


Figure 2a. AlexNet without normalization.

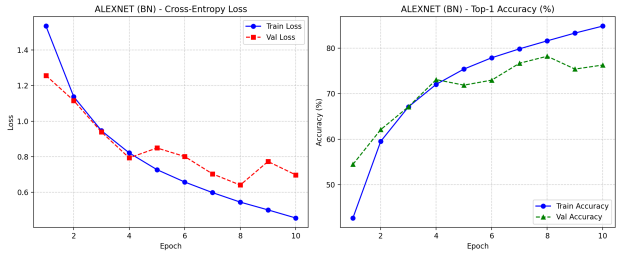


Figure 2b. AlexNet with BatchNorm.

5. Data Augmentation and Generalization

Configuration	Final train acc.	Final val. acc.	Generalization gap Δ	Best checkpoint acc.	Macro F1
BN, no augmentation	90.34%	82.14%	8.20 pp	82.42%	82.44%
BN, crop + flip + jitter	84.38%	79.73%	4.65 pp	81.26%	81.17%

Augmentation reduced the final generalization gap from 8.20 to 4.65 percentage points, a 3.56-point contraction. It did not improve the best 15-epoch validation score: 81.26% versus 82.42% without augmentation (−1.16 pp). The correct interpretation is not that augmentation failed. The stochastic crop/color distribution makes each epoch harder and roughly doubled mean epoch time (52.56 s versus 23.48 s), while the lower training accuracy and smaller gap show reduced memorization. More epochs or a tuned learning-rate schedule would be needed to test whether the stronger invariances eventually exceed the unaugmented checkpoint.

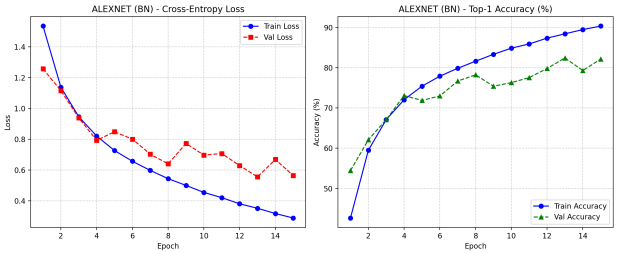


Figure 3a. AlexNet-BN without augmentation.

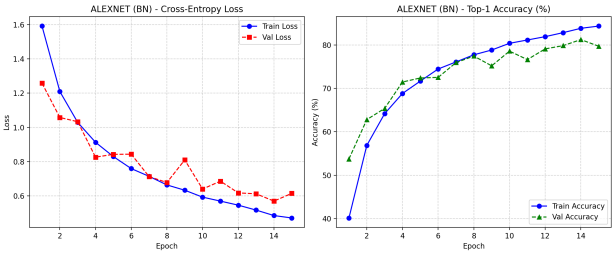


Figure 3b. AlexNet-BN with augmentation.

Label-preserving transformations alter position, crop boundary, and photometric appearance while retaining class identity. The network therefore cannot depend as strongly on canonical pixel placement or color statistics and must learn features that remain useful across plausible views.

6. Dual-Dataset Benchmark

On Tiny ImageNet-200, AlexNet-BN with augmentation reached 28.91% top-1 accuracy and 26.94% macro F1. Random guessing is 1/200 = 0.50%, so the model is 57.8× chance in absolute accuracy. This confirms that the network learned class-discriminative features despite the harder label space.

Dataset	Classes	Validation accuracy	Macro F1	Chance accuracy
CIFAR-10	10	81.26%	81.17%	10.00%
Tiny ImageNet-200	200	28.91%	26.94%	0.50%

The completed seven-model Tiny ImageNet ladder ranged from 0.50% (AlexNet + LRN) to 30.63% (AlexNet + BatchNorm). This wide spread under a fixed 10-epoch budget shows that optimization behavior and inductive bias matter at least as much as nominal parameter count when the class space expands from 10 to 200.

Tiny ImageNet is substantially harder because it contains 200 often fine-grained categories, more varied backgrounds and viewpoints, and fewer examples per class. Its 64×64 resolution also compresses small discriminative details. The resulting macro F1 (26.94%) trails accuracy (28.91%), indicating uneven per-class recall/precision rather than uniform performance. The saved Tiny ImageNet curve should be read for early plateaus and widening train/validation separation; a longer schedule alone may not be sufficient without stronger regularization or a better architecture.

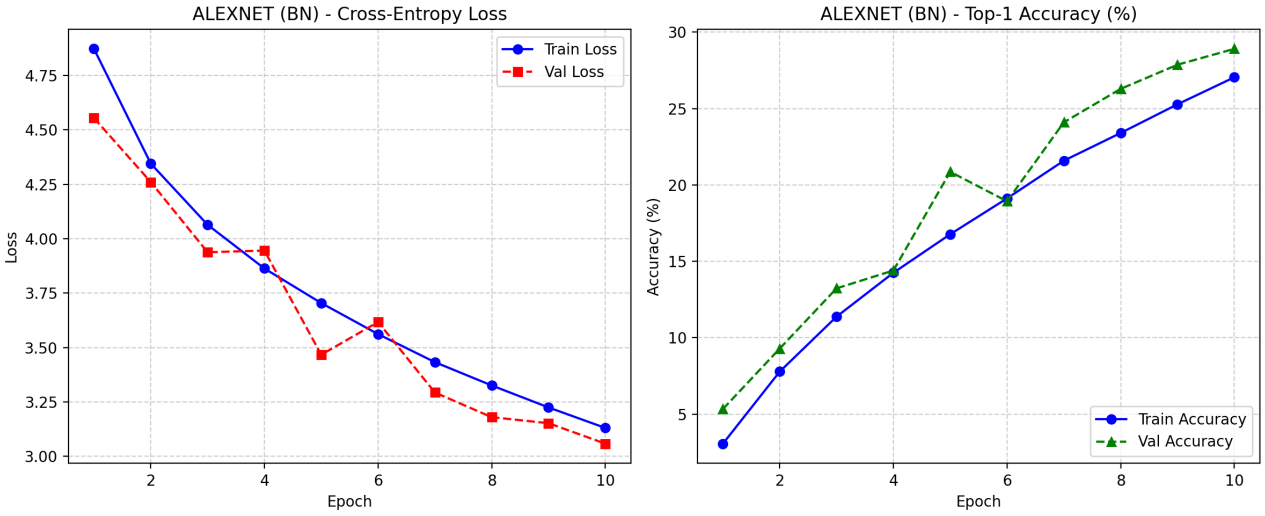


Figure 4. AlexNet-BN training dynamics on Tiny ImageNet-200.

7. Exploratory Data Analysis

Class-count plots verify the intended split structure, while channel histograms expose dataset-level intensity statistics that motivate normalization. CIFAR-10 is balanced by construction. Tiny ImageNet's training set is balanced at 500 images per class; the validation annotations provide 50 images per class.

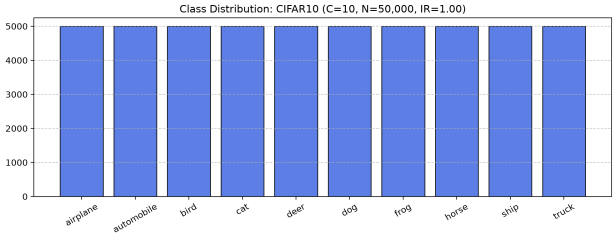


Figure 5a. CIFAR-10 class distribution.

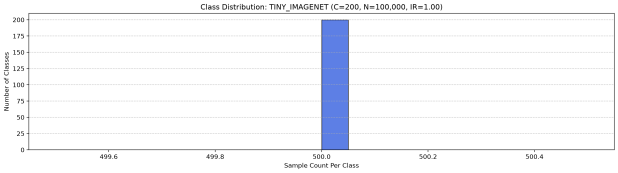


Figure 5b. Tiny ImageNet class distribution.

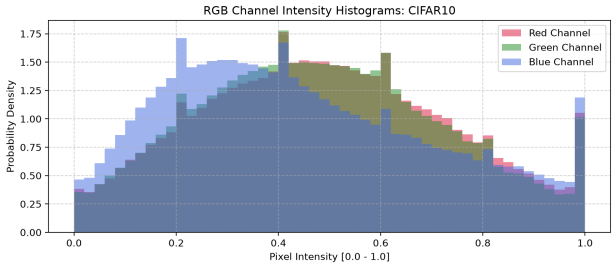


Figure 6a. CIFAR-10 RGB feature distributions.

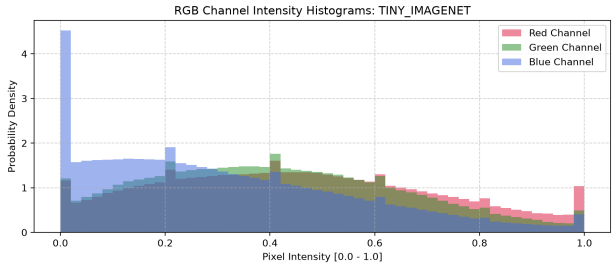


Figure 6b. Tiny ImageNet RGB feature distributions.

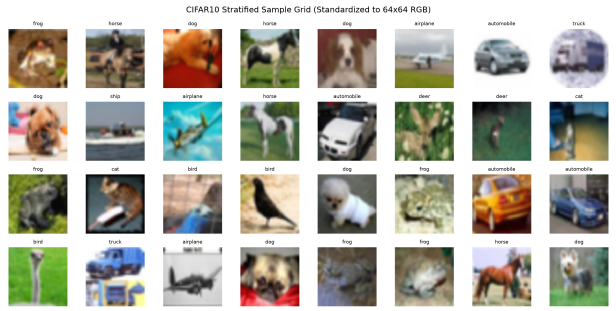


Figure 7a. CIFAR-10 samples.



Figure 7b. Tiny ImageNet samples.

8. Transform Pipeline Diagnostic

The diagnostic below confirms the semantic difference among NumPy/OpenCV and PyTorch representations. OpenCV and NumPy retain channels-last HWC arrays in CPU memory. PyTorch converts to a channels-first CHW floating-point tensor and applies dataset standardization; on Colab the same tensor pipeline can remain resident in GPU memory after transfer.

```
[1] OpenCV output
type: numpy.ndarray   dtype: float32   shape: (64, 64, 3) [H,W,C]
range: [0.0000, 1.0000]   memory: host RAM   color: BGR -> RGB

[2] NumPy output
type: numpy.ndarray   dtype: float32   shape: (64, 64, 3) [H,W,C]
range: [0.0000, 1.0000]   memory: host RAM

[3] PyTorch/torchvision output
type: torch.Tensor   dtype: torch.float32   shape: [3, 64, 64] [C,H,W]
range: approximately [-1.99, 2.13] after standardization
local diagnostic memory: CPU RAM; Colab training tensors: Tesla T4 VRAM

Local 1,000-image diagnostic (illustrative; hardware-dependent)
OpenCV CPU: 24.55 ms | NumPy CPU: 38.57 ms | PyTorch CPU: 358.95 ms
```

These microbenchmarks measure small individual transforms and should not be treated as end-to-end training throughput. GPU tensor transforms become attractive when batched and when avoiding repeated host-to-device transfer; CPU vectorized libraries can be faster for a single 64×64 sample.

9. Model Profiling Output

Model	Params(M)	MACs(M)	FLOPs(M)	Latency(ms)	Throughput(img/s)
LeNet-5 (Base)	0.338	3.59	7.17	0.39	2591.9
LeNet-Modern	0.446	4.59	9.19	0.66	1518.9
AlexNet-64 (None)	6.609	218.35	436.70	2.12	472.2
AlexNet-64 (LRN)	6.609	218.35	436.70	1.94	515.0
AlexNet-64 (BatchNorm)	6.611	218.98	437.97	1.39	717.7
Torchvision Zoo AlexNet	57.045	94.10	188.20	2.15	466.0

Device: Tesla T4 | Input: (1, 3, 64, 64) | FLOPs ≈ 2 × MACs

The zoo model's lower convolutional FLOP count but much larger parameter count illustrates that parameter storage and arithmetic cost are distinct. Dense layers dominate zoo parameters, whereas convolutions dominate arithmetic. Latency is also affected by kernel implementations, memory traffic, and launch overhead; it is not a monotonic function of FLOPs.

10. Conclusions

1. Modernization: ReLU and overlapping max pooling improved five-epoch LeNet accuracy by 11.67 points and reduced the generalization gap slightly.

2. Normalization: BatchNorm was decisively better than no normalization and LRN, reaching 70% validation accuracy three epochs sooner and adding little compute.

3. Augmentation: The stochastic pipeline reduced the generalization gap by 3.56 points, but the 15-epoch budget was too short to exceed the unaugmented best checkpoint.

4. Dataset difficulty: Tiny ImageNet accuracy remained far above chance but far below CIFAR-10 because the class count and visual diversity are much greater.

5. Architecture fit: The compact manual AlexNet-64 outperformed the much larger torchvision baseline on this 64×64 task, showing that parameter count alone does not guarantee better generalization.

The experiments support a coherent engineering conclusion: architectural choices that improve gradient propagation and task-specific spatial design matter more than simply increasing parameter count. BatchNorm supplied the strongest short-horizon optimization benefit, while augmentation traded immediate fit for better regularization.

11. Reproducibility and Submission Contents

Artifact	Purpose
models/, utils/, configs/	Architectures, data pipeline, metrics, visualization, profiling, and seeding
train.py / evaluate.py	Training and checkpoint evaluation entry points
run_experiments.py	Reproducible experiment matrix with seed-aware output names
commands.txt	Exact CLI commands for all required runs
checkpoints/	Selected Base LeNet, Modern LeNet, and AlexNet-BN checkpoints
plots/ and results/	EDA, curves, JSON metrics, and generated tables
github_repo.txt	Local repository status and eventual remote URL

Execution environment: Google Colab with an NVIDIA Tesla T4 GPU; exact Python/PyTorch dependencies are recorded in requirements.txt. Primary random seed: 42; supplemental seeds: 123 and 2026. The data/ directory is intentionally excluded from version control and the submission archive.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” Proceedings of the IEEE, 1998.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” NeurIPS, 2012.

[3] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.

[4] K. He et al., “Deep Residual Learning for Image Recognition,” CVPR, 2016.

[5] A. G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” 2017.